NoSQL Databases

Francesco Pelacani
2025-2026

Graph Databases

What is a Graph Database?

Graph databases allow you to store entities and relationships between these entities. Entities are also referred as nodes, which have properties. Relations are known as edges that can have properties.

Edge have directional significance; nodes are organized by relationships which allow the discovery of patterns between nodes.

Features

- **Consistency** → Usually single server, full ACID-compliant.
- **Transactions** → Supported, writes requires the start of a transaction
- **Availability** → Master-slave replication can be used to improved it
- **Query features** → Can query the data using a dedicated Query Language (Cypher)
- **Scaling** → Usually adopted strategy is scale-up

Usage

Suitable Use Cases

1. Social Networks (connected data)
2. Location-Based Services
3. Recommendation Engines

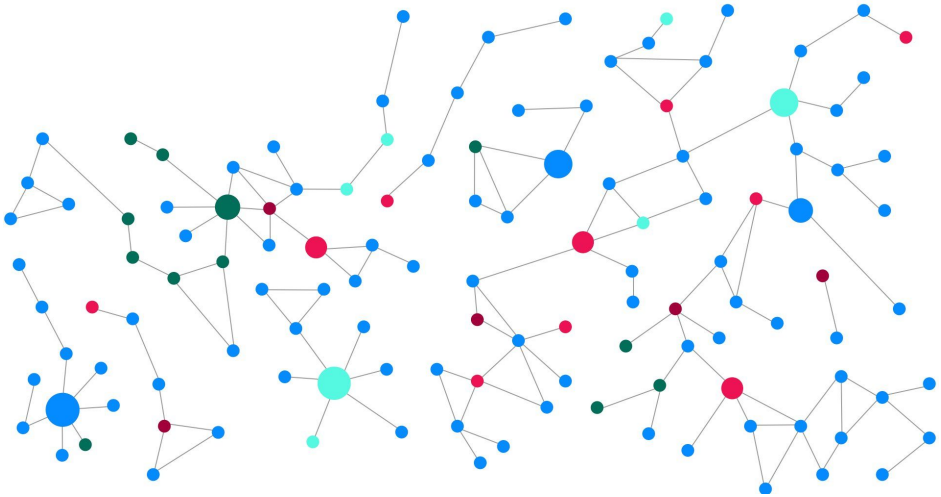
When Not to Use

1. When you want to perform massive update over your dataset

Neo4j

What is a Graph?

A graph shows how objects are related to each other.
Neo4j uses the **graph structure** to store data and is known as a **labeled property graph**.



How is data organized?

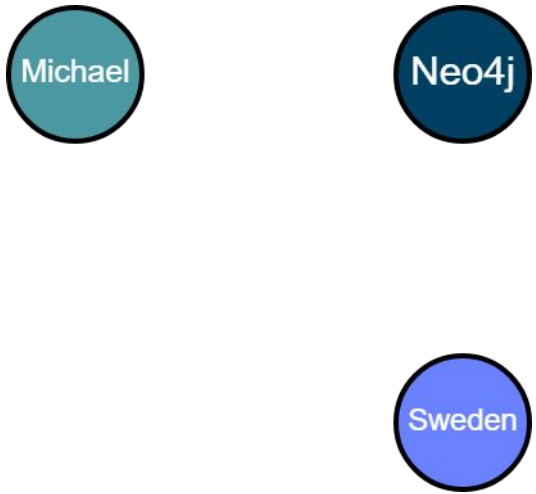
Neo4j stores and organizes data using these four elements:

- **Nodes** - The fundamental entities in the graph
- **Labels** - Categories or types for nodes
- **Relationships** - The connections between nodes
- **Properties** - The attributes of nodes and relationships

Nodes

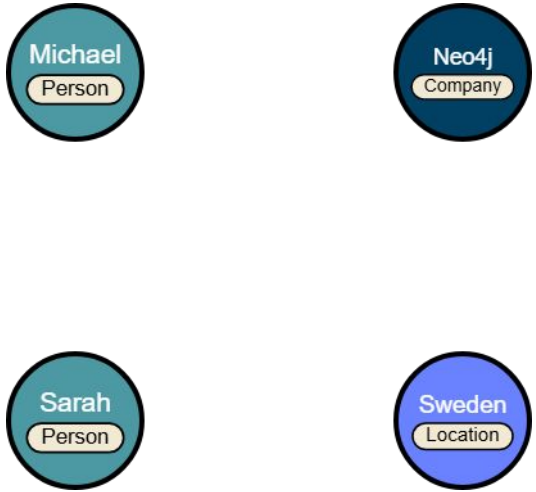
Nodes are the circles in a graph. They **represent objects or entities** in your data model.

Imagine a social network, the entities (e.g. people, locations, companies) would be represented by nodes.



Labels

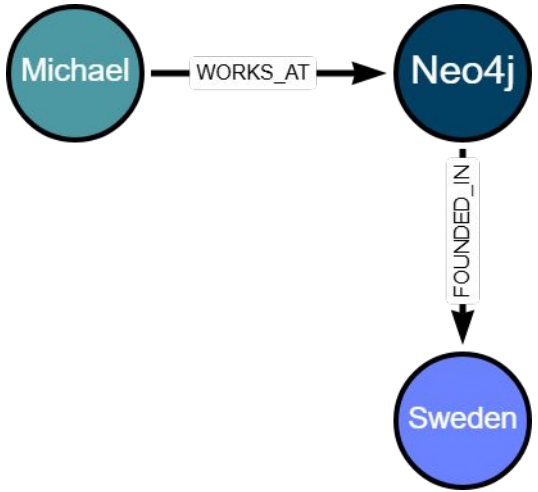
Nodes are grouped by or categorized using labels. A node can have multiple labels.



Labels describe what the nodes are. Should be a singular noun.

Relationships

Relationships are the lines in the graph. **Relationships describe how nodes within the graph are connected to each other**.

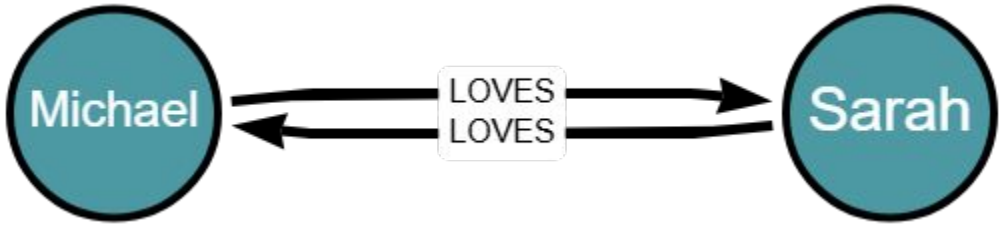


A relationship connects two nodes, start and end. All relationships have a **type** and a **direction**. Use verbs for relationship types.

Multiple Relationships

Nodes can have multiple relationships to other nodes.

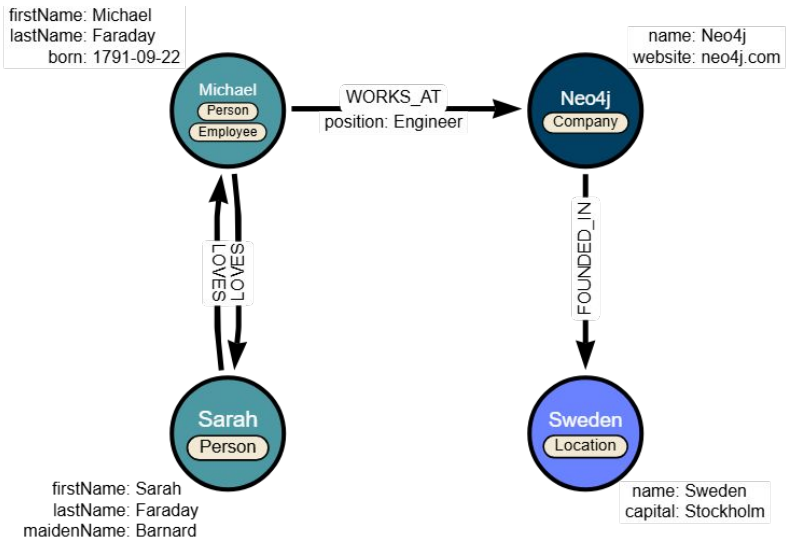
Multiple relationships can be used to describe **bi-directional relationships**.



Properties

You can store data against nodes and relationships as properties.

Properties are named key (unique), typed (int, bool, string, list, ..) value pairs.



Why Graph Databases?

Relationships in a graph are treated with the same importance as nodes that connect them. This is not the same as other database technologies, that can have issues with:

- Poorly performing queries when traversing complex schema of foreign keys and many-to-many relationships.
- Dealing with hierarchical data or trees, where the answer may lie at an unknown or varying depth.
- Finding a path through an ever-changing dataset with complex requirements.
- Simply finding it difficult to express what you are looking for.

These issues are often as a result of storing data in tables when the requirement is to understand relationships not rows.

The O(n) Problem (I)

Tabular data is a tried-and-tested methodology that works well for many use cases and an ecosystem of tooling exists to help you work efficiently.

However, **as the amount of data grows or the application or use case becomes more complex**, you may encounter challenges dealing with relationships.

When querying across tables, the joins are computed at read-time, using an index to find the corresponding rows in the target table. The more data added to the database, the larger the index grows, the slower the response time.

The O(n) Problem (II)

The Rise of NoSQL

Over the years, many No-SQL databases have sprung up to handle different scenarios, for example:

- **Document** stores offer flexibility.
- **Wide-column stores** offer scalability for large datasets.
- **Key-value stores** provide simplicity and high performance.
- **Graph databases** enable efficient modeling and querying of complex relationships.

How does a Graph Database reason?

When you create a relationship between two nodes, the database stores a **pointer to the relationship with each node**.

When reading data, the database will follow pointers in memory rather than relying on an underlying index.

This means that the query time remains constant to the size of the relationships expanded regardless of the overall size of the data.

Graph Query Language

How to query a Graph Database?

Graph databases have their own query language, **GQL**, an ISO standard for graph databases.

Cypher is Neo4j's implementation of GQL.

Cypher is a **declarative language**, meaning the database is responsible for finding the most optimal way of executing that query.

Patterns

Cypher is a declarative query language that allows you to identify **patterns** in your data using an **ASCII-art style syntax** consisting of **brackets**, **dashes** and **arrows**.

```
(p:Person)-[r:ACTED_IN]→(m:Movie)
```

Nodes

Nodes in the pattern are expressed with parentheses - **( )**

```
(p:Person)-[r:ACTED_IN]→(m:Movie)
```

Inside the parentheses you can define information about the node, for example the label(s) or properties the node should contain. Labels are prefixed a colon - **(:Label)**

```
(p:Person)-[r:ACTED_IN]→(m:Movie)
```

Relationships

Relationships are drawn with two dashes and an arrow to specify the direction - **- ->**

```
(p:Person)-[r:ACTED_IN]→(m:Movie)
```

Relationship information is contained within square brackets, and its type is prefixed with a colon - **[:TYPE]**

```
(p:Person)-[r:ACTED_IN]→(m:Movie)
```

Variables

The nodes and relationships in the pattern are assigned to variables.

```
(p:Person)-[r:ACTED_IN]→(m:Movie)
```

These variables are positioned before the information about the node or relationship:

- **p** - the :Person node
- **r** - the :ACTED_IN relationship
- **m** - the :Movie node

MATCH-ing

The **MATCH** clause is used to find patterns in the data.

```
MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)
WHERE p.name = 'Tom Hanks'
RETURN p,r,m
```

```
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)<-[r:ACTED_IN]-(p2:Person)
WHERE p.name = 'Tom Hanks'
RETURN p2.name AS actor, m.title AS movie, r.role AS role
```

The keyword **AS** is used to define an alias (like in SQL).

Creating Graphs

To **create nodes** in the database, you use the **MERGE** clause.

```
MERGE (m:Movie {title: "Arthur the King"})
SET m.year = 2024
RETURN m
```

Relationships are created by expressing a **pattern that connects two nodes.**

```
MERGE (m:Movie {title: "Arthur the King"})
MERGE (u:User {name: "Adam"})
MERGE (u)-[r:RATED {rating: 5}]->(m)
RETURN u, r, m
```

Practice - I

practice

Complete the exercise on Github

practice

Extra Neo4j Material

Neo4j GraphAcademy